

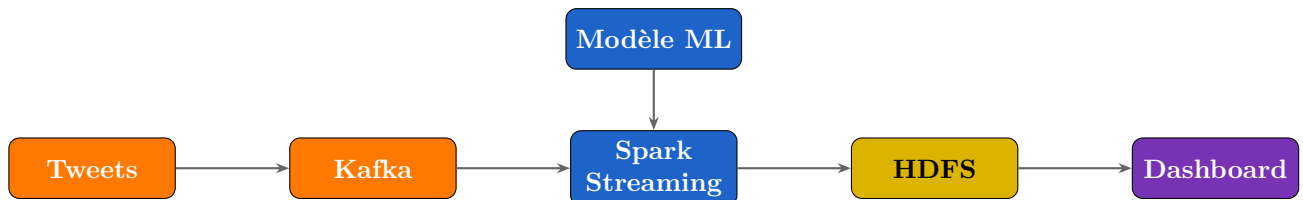
Projet

Analyse de Sentiment en Temps Réel sur les Réseaux Sociaux

Module : Approfondissement en Big Data

Formation : BUT 3 Science de Données — Université de Poitiers

Année universitaire : 2025–2026



Consignes générales

- **Livrables :** notebooks complétés 01, 02, 03, **dashboard** + modèle ML sauvegardé sur HDFS + réponses aux questions ouvertes.
- Les notebooks sont à compléter directement sur le cluster.
- Barème total : **/20 points** (détail dans chaque section).

Table des matières

1	Contexte et objectifs	2
1.1	Scénario	2
1.2	Organisation du projet	2
1.3	Architecture générale	2
2	Environnement technique	3
2.1	Accès au cluster	3
2.2	Infrastructure du cluster	3
2.3	Données	3
3	Phase 1 — Partie A : Indicateurs Streaming	4
4	Phase 1 — Partie B : Modèle de Sentiment	7
5	Phase 2 — Mise en production	11
6	Dashboard temps réel	14
7	Questions ouvertes — Analyse et originalité	15
8	Livrables et rendu	16
8.1	Liste des livrables	16
8.2	Modalités de rendu	16
Annexes		17
A.	Aide-mémoire Spark Structured Streaming	17
B.	Aide-mémoire Apache Kafka	17
C.	Commandes HDFS utiles	18

1 Contexte et objectifs

1.1 Scénario

*Vous venez d'être recrutés par **SentWatch**, une startup spécialisée dans la veille des réseaux sociaux. Votre première mission : concevoir un système capable d'analyser le sentiment de millions de tweets en temps réel.*

Le client final est une grande marque qui souhaite surveiller sa réputation en ligne. Elle veut savoir, seconde par seconde, si les conversations autour de sa marque sont positives ou négatives, et être alertée immédiatement en cas de crise.

1.2 Organisation du projet

Le projet se décompose en **deux phases** complémentaires :

Phase 1 — Développement Construire les briques séparément dans des notebooks.

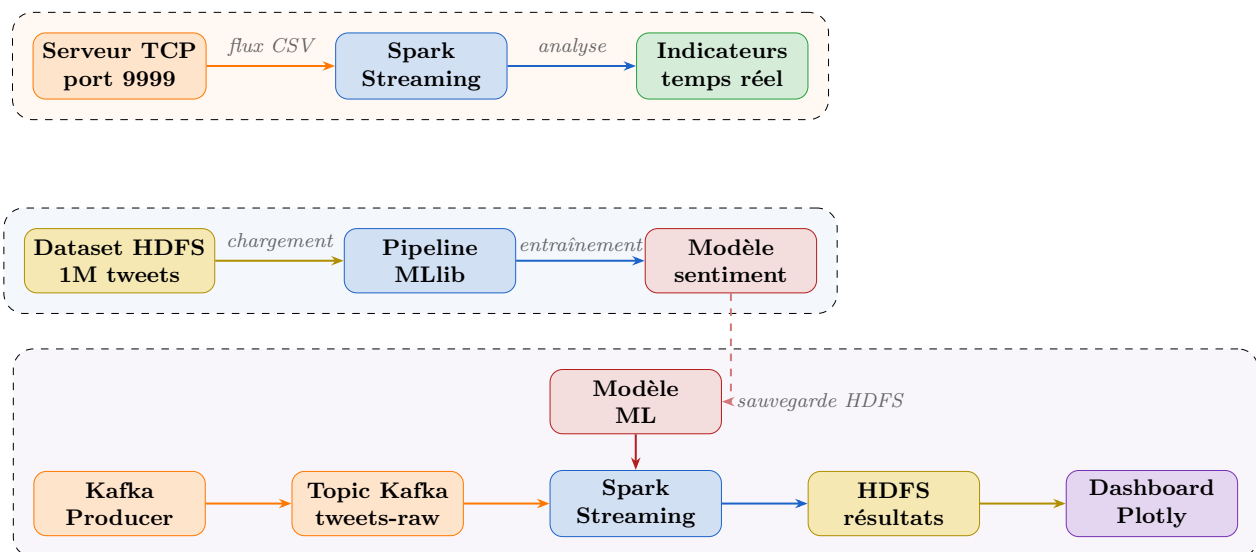
- **Track A — Indicateurs streaming** (§3) : sur des tweets pré-tagués (sentiment connu), développer des indicateurs d'analyse en temps réel (volume, ratio de sentiment, hashtags trending, alertes...).
- **Track B — Modèle ML** (§4) : entraîner un modèle de classification capable de prédire le sentiment d'un tweet à partir de son texte.

Phase 2 — Production Assembler le tout en un pipeline temps réel complet :

Kafka → Spark Structured Streaming + ML → HDFS → Dashboard

1.3 Architecture générale

Le diagramme ci-dessous présente l'architecture complète du système, de l'ingestion des données brutes jusqu'à la visualisation.



2 Environnement technique

2.1 Accès au cluster

- **JupyterHub** : <http://10.15.61.20:8888> — connectez-vous avec vos **identifiants LDAP universitaires**.
- **Kafka-UI** : <http://10.15.61.20:8080> — interface web pour visualiser vos topics Kafka.

Premier réflexe

Dès votre première connexion, ouvrez et exécutez le notebook `00_Setup.ipynb`. Ce notebook vérifie que votre environnement est correctement configuré (accès HDFS, Spark, Kafka) et crée les répertoires nécessaires.

2.2 Infrastructure du cluster

Composant	Détail
Nœuds	1 master + 8 workers
Hadoop	Version 3.2.4
Spark	Version 3.5.3
Gestionnaire de ressources	YARN
Kafka	Accessible via <code>kafka:9092</code>

2.3 Données

Les données sont disponibles sur HDFS :

Chemin HDFS	Description	Taille
<code>hdfs:///data/tweets/tweets_dev.csv</code>	Tweets pour le développement (Phase 1)	1 000 000 tweets
<code>hdfs:///data/tweets/tweets_prod.csv</code>	Tweets pour la production (Phase 2)	600 000 tweets

Format CSV Le fichier est **sans en-tête** et contient **6 colonnes** :

#	Colonne	Description
1	<code>sentiment</code>	0 = négatif, 4 = positif
2	<code>tweet_id</code>	Identifiant unique du tweet
3	<code>date</code>	Date de publication
4	<code>query</code>	Requête de collecte (souvent <code>NO_QUERY</code>)
5	<code>user</code>	Nom d'utilisateur
6	<code>text</code>	Contenu textuel du tweet

Format d'exemple

```
0,1467810369,"Mon May 11 03:30:24 UTC 2009",NO_QUERY,scotthamilton,"is upset that..."
```

3 Phase 1 — Partie A : Indicateurs Streaming

/5 points

Contexte

Le serveur TCP est lancé par l'enseignant sur le **port 9999**. Il envoie un flux de tweets pré-tagués (le sentiment est déjà connu). Votre objectif est de construire des **indicateurs d'analyse en temps réel** à partir de ce flux.

Notebook : 01_Streaming_Indicateurs_etudiant.ipynb

Exercice 3.1 — Parsing du flux

/1 pt

Consigne : Connectez-vous au flux TCP (`host="master", port=9999`) et parsez chaque ligne CSV reçue. Convertissez le sentiment : $0 \rightarrow -1$ (négatif), $4 \rightarrow +1$ (positif). Ajoutez un *timestamp* d'événement.

Indice

```
split(col("value"), ",")
regexp_replace(...)
current_timestamp()
```

Exercice 3.2 — Volume par fenêtre temporelle

/0.5 pt

Consigne : Comptez le nombre de tweets reçus par fenêtre de **30 secondes**.

Indice

```
groupBy(window("event_time", "30 seconds")).count()
```

Exercice 3.3 — Ratio de sentiment par fenêtre

/0.5 pt

Consigne : Pour chaque fenêtre de 30 secondes, calculez le ratio de tweets positifs et négatifs.

Indice

```
avg("sentiment_score")
# ou bien :
sum(when(col("sentiment_score") == 1, 1)) / count("*")
```

Exercice 3.4 — Hashtags trending/1 pt

Consigne : Extrayez les hashtags de chaque tweet et classez-les par popularité dans une fenêtre de **1 minute**.

Indice

```
regexp_extract_all(col("text"), lit(r"#(\w+)"), 1)
explode(...)
groupBy(window("event_time", "1 minute"), "hashtag").count()
```

Exercice 3.5 — Mentions les plus fréquentes/0.5 pt

Consigne : Même exercice que le précédent, mais avec les **mentions** @utilisateur. Extrayez les mentions de chaque tweet et classez-les par popularité.

Exercice indépendant : pas d'indice détaillé.

Exercice 3.6 — Analyse des utilisateurs/0.5 pt

Consigne : Identifiez les utilisateurs les plus actifs et leur sentiment moyen.

Indice

```
groupBy("user").agg(
  count("*").alias("nb_tweets"),
  avg("sentiment_score").alias("sentiment_moyen")
)
```

Exercice 3.7 — Système d'alerte/0.5 pt

Consigne : Implémentez une alerte qui se déclenche quand le **sentiment moyen chute en dessous de -0.3** dans une fenêtre glissante de **1 minute** avec un pas de **15 secondes**.

Indice

```
window("event_time", "1 minute", "15 seconds")
.filter(col("avg_sentiment") < -0.3)
```

Exercice 3.8 — Watermarking/0.5 pt

Consigne : Ajoutez le *watermarking* à l'une de vos requêtes pour gérer les données tardives. Expliquez **en commentaire** l'effet du watermark sur votre requête.

Indice

```
.withWatermark("event_time", "1 minute")
```

Le watermark indique à Spark qu'il peut abandonner l'état des fenêtres dont le temps est inférieur à `max_event_time - 1 minute`.

4 Phase 1 — Partie B : Modèle de Sentiment

/5 points

Contexte

Vous disposez d'un dataset de **1 million de tweets** avec leur sentiment. Votre objectif est de construire un modèle de **classification binaire** capable de prédire le sentiment d'un tweet à partir de son contenu textuel.

Notebook : 02_ML_Sentiment_Model_etudiant.ipynb

Exercice 4.1 — Chargement et exploration

/0.5 pt

Consigne : Chargez le dataset depuis HDFS. Renommez les colonnes conformément au format décrit en section 2. Affichez la distribution des sentiments, le nombre total de tweets et quelques exemples. Convertissez les labels : 4 → 1.0 (positif), 0 → 0.0 (négatif).

Indice

```
df = spark.read.csv("hdfs:///data/tweets/tweets_dev.csv", header=False)
df = df.withColumnRenamed("_c0", "sentiment") \
      .withColumnRenamed("_c5", "text") # etc.
df = df.withColumn("label", when(col("sentiment") == 4, 1.0)
                      .otherwise(0.0))
```

Exercice 4.2 — Nettoyage du texte

/1 pt

Consigne : Créez une **UDF Python** qui nettoie le texte d'un tweet en appliquant les transformations suivantes (dans l'ordre) :

1. Passage en **minuscules**
2. Suppression des **URLs** (pattern : `http\S+`)
3. Suppression des **mentions** (pattern : `@\w+`)
4. Suppression du symbole **#** (garder le mot qui suit)
5. Suppression des **caractères non-alphabétiques** (sauf les espaces)
6. Normalisation des espaces (pas de double espace)

Indice

```
import re
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def clean_text(text):
    if text is None:
        return ""
```



```

text = text.lower()
text = re.sub(r"http\S+", "", text)
text = re.sub(r"@w+", "", text)
text = re.sub(r"#", "", text)
text = re.sub(r"[^a-z\s]", "", text)
text = re.sub(r"\s+", " ", text).strip()
return text

clean_udf = udf(clean_text, StringType())

```

Important — Training-Serving Skew

Conservez cette UDF précieusement. Vous devrez appliquer **exactement le même nettoyage** en Phase 2 lors de l'inférence en streaming. Toute différence entre le prétraitement à l'entraînement et celui en production dégradera les performances du modèle.

Exercice 4.3 — Construction du pipeline ML

/1.5 pt

Consigne : Construisez un **pipeline PySpark MLlib** avec les étapes suivantes :

1. `RegexTokenizer` — découpage du texte en tokens
2. `StopWordsRemover` — suppression des mots vides (anglais)
3. `CountVectorizer` — construction du vocabulaire (`vocabSize=10000`)
4. `IDF` — pondération TF-IDF
5. `LogisticRegression` — classification binaire (`maxIter=10`)

Indice

```

from pyspark.ml.feature import (RegexTokenizer, StopWordsRemover,
                                CountVectorizer, IDF)
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

tokenizer = RegexTokenizer(inputCol="clean_text",
                           outputCol="tokens", pattern="\\W")
remover   = StopWordsRemover(inputCol="tokens",
                              outputCol="filtered")
cv        = CountVectorizer(inputCol="filtered",
                             outputCol="raw_features",
                             vocabSize=10000)
idf       = IDF(inputCol="raw_features", outputCol="features")
lr        = LogisticRegression(featuresCol="features",
                                labelCol="label", maxIter=10)

pipeline = Pipeline(stages=[tokenizer, remover, cv, idf, lr])

```

Exercice 4.4 — Entraînement et évaluation

/1.5 pt

Consigne : Divisez les données en **train (80 %)** et **test (20 %)**. Entraînez le modèle. Évaluez-le avec les métriques suivantes :

- **AUC-ROC** via `BinaryClassificationEvaluator`
- **Accuracy, F1-score, Precision, Recall** via `MulticlassClassificationEvaluator`
- **Matrice de confusion** via `crosstab`

Objectif de performance

Votre modèle doit atteindre un **AUC** > 0.70 . Si ce n'est pas le cas, vérifiez votre pipeline de nettoyage et ajustez les hyperparamètres.

Indice

```
train_df, test_df = df_clean.randomSplit([0.8, 0.2], seed=42)
model = pipeline.fit(train_df)
predictions = model.transform(test_df)

from pyspark.ml.evaluation import (BinaryClassificationEvaluator,
                                   MulticlassClassificationEvaluator)
auc = BinaryClassificationEvaluator(labelCol="label") \
    .evaluate(predictions)
accuracy = MulticlassClassificationEvaluator(
    labelCol="label", metricName="accuracy") \
    .evaluate(predictions)
```

Exercice 4.5 — Sauvegarde du modèle

/0.25 pt

Consigne : Sauvegardez le modèle entraîné sur HDFS au chemin **exact** suivant :

`hdfs:///models/sentiment_model`

Ce chemin sera utilisé en Phase 2 pour charger le modèle en production.

```
model.write().overwrite().save("hdfs:///models/sentiment_model")
```

Exercice 4.6 — Vérification

/0.25 pt

Consigne : Rechargez le modèle depuis HDFS et testez-le sur **5 tweets de votre choix**. Vérifiez que les prédictions sont cohérentes.

Indice

```
from pyspark.ml import PipelineModel
loaded_model = PipelineModel.load("hdfs:///models/sentiment_model")
# Créer un petit DataFrame avec 5 tweets, appliquer clean_udf,
```

```
# puis loaded_model.transform(...)
```

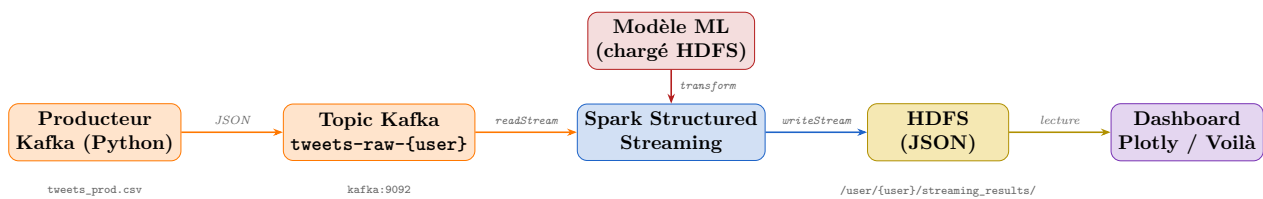
5 Phase 2 — Mise en production

/4 points

Contexte

Tout ce que vous avez développé séparément doit maintenant fonctionner **ensemble en temps réel**. L'architecture cible : votre producteur Kafka envoie les 600 000 tweets restants → Spark lit depuis Kafka et applique le modèle ML → les résultats sont écrits sur HDFS → le dashboard les affiche.

Architecture de la phase de production



Notebook : 03_Production_Pipeline_etudiant.ipynb

Exercice 5.1 — Producteur Kafka minimal

/0.5 pt

Consigne : En utilisant la bibliothèque `kafka-python`, écrivez un petit producteur qui envoie 10 tweets au format JSON vers votre topic Kafka personnel `tweets-raw-{username}`.

Indice

```

from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='kafka:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
topic = f"tweets-raw-{username}"
for tweet in sample_tweets:
    producer.send(topic, value=tweet)
producer.flush()
  
```

Exercice 5.2 — Consommateur Kafka minimal

/0.5 pt

Consigne : Écrivez un consommateur qui lit les messages depuis votre topic et les affiche.

Indice

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    topic,
    bootstrap_servers='kafka:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)
for message in consumer:
    print(message.value)
    # Arrêter après quelques messages pour le test
```

Exercice 5.3 — Spark Structured Streaming depuis Kafka**/1 pt**

Consigne : Configurez Spark pour lire en streaming depuis votre topic Kafka. Parsez le JSON contenu dans la colonne value.

Indice

```
df_kafka = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", f"tweets-raw-{username}") \
    .load()

from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StringType

schema = StructType() \
    .add("tweet_id", StringType()) \
    .add("text", StringType()) \
    .add("user", StringType()) \
    .add("date", StringType())

df_parsed = df_kafka.select(
    from_json(col("value").cast("string"), schema).alias("data")
).select("data.*")
```

Exercice 5.4 — Intégration du modèle ML**/1 pt**

Consigne : Chargez le modèle sauvegardé en Phase 1. Appliquez le même nettoyage de texte (UDF identique à celui de l'entraînement). Appliquez le modèle sur le flux streaming.

Training-Serving Skew

Le nettoyage du texte doit être **strictement identique** à celui utilisé lors de l'entraînement (exercice 4.2). Toute différence (même minime) peut dégrader fortement les prédictions.

Indice

```
from pyspark.ml import PipelineModel

model = PipelineModel.load("hdfs:///models/sentiment_model")
df_cleaned = df_parsed.withColumn("clean_text", clean_udf("text"))
df_predictions = model.transform(df_cleaned)
```

Exercice 5.5 — Écriture des résultats**/0.5 pt**

Consigne : Écrivez les prédictions vers HDFS au format JSON.

Indice

```
query = df_predictions \
    .select("tweet_id", "user", "text", "prediction") \
    .writeStream \
    .format("json") \
    .option("path",
            f"hdfs:///user/{username}/streaming_results/") \
    .option("checkpointLocation",
            f"hdfs:///user/{username}/checkpoints/prod") \
    .outputMode("append") \
    .start()
```

Exercice 5.6 — Pipeline complet**/0.5 pt**

Consigne : Lancez le producteur Kafka depuis un **terminal JupyterHub** :

```
python ~/notebooks/scripts/kafka_producer.py --rate 50
```

Vérifiez dans **Kafka-UI** (<http://10.15.61.20:8080>) que les messages arrivent dans votre topic. Vérifiez ensuite que les résultats apparaissent dans HDFS :

```
hdfs dfs -ls /user/$USER/streaming_results/
hdfs dfs -cat /user/$USER/streaming_results/*.json | head -5
```

6 Dashboard temps réel

/2 points

Notebook : `dashboard_etudiant.ipynb`

Consigne : Créez un notebook-dashboard qui lit les résultats depuis HDFS et les visualise avec **Plotly**. Le dashboard doit contenir **au minimum 3 visualisations** parmi :

- **KPIs** : total de tweets analysés, pourcentage positif, pourcentage négatif.
- **Évolution du sentiment** dans le temps (line chart).
- **Top 10 utilisateurs** les plus actifs (bar chart).
- **Mots les plus fréquents** dans les tweets positifs vs négatifs (bar chart).
- **Tableau des tweets récents** avec leur sentiment prédit.
- **Alerte visuelle** si le sentiment moyen est très négatif.

Rafraîchissement automatique

Le dashboard doit se rafraîchir automatiquement. Utilisez une boucle avec `clear_output()` et `time.sleep()` pour mettre à jour les graphiques périodiquement.

Le dashboard peut également être servi via **Voilà** :

<http://10.15.61.20:8888/user/{username}/voila/render/notebooks/dashboard.ipynb>

7 Questions ouvertes — Analyse et originalité

Questions ouvertes – Analyse et originalité /4 points

Ces questions évaluent votre capacité d'analyse critique, votre créativité et votre compréhension en profondeur de l'architecture. Répondez de manière argumentée, avec des exemples concrets. L'implémentation (quand demandée) rapporte des points supplémentaires.

7.1 – Analyse critique

/1 pt

- Limites du bag-of-words / TF-IDF.** Quelles sont les limites de l'approche bag-of-words / TF-IDF pour l'analyse de sentiment ? Donnez **au moins 3 exemples concrets** de tweets que votre modèle classerait probablement mal, et expliquez pourquoi.
- Obsolescence du dataset.** Le dataset Sentiment140 date de 2009. Quels problèmes cela pose-t-il pour une utilisation en 2026 ? Proposez des solutions concrètes.

7.2 – Conception d'un système d'alerte intelligent

/1 pt

Une marque vous demande de détecter les « crises de réputation » en temps réel. Concevez un système d'alerte qui :

- Définit des **critères de déclenchement pertinents** (pas un simple seuil fixe).
- **Minimise les faux positifs.**
- **S'adapte** au volume normal de la marque.

Décrivez votre approche et, si possible, implémentez-la dans un notebook.

7.3 – Indicateur original

/1 pt

Proposez et implémentez un **indicateur streaming** qui n'est pas demandé dans les exercices. Exemples (non exhaustifs) :

- | | |
|---------------------------------------|--|
| — Détection de bots | — Score de polarisation |
| — Vitesse de propagation d'un hashtag | — Corrélation activité / sentiment par tranche horaire |

Justifiez l'intérêt de votre indicateur et interprétez les résultats.

7.4 – Réflexion architecturale

/1 pt

- TCP vs Kafka.** Comparez le socket TCP (Phase 1) et Kafka (Phase 2) : avantages, inconvénients, cas d'usage.
- Back-pressure.** Que se passe-t-il si le consumer Spark est plus lent que le producer ? Comment Kafka gère-t-il cette situation par rapport à TCP ?
- Passage à l'échelle.** Comment passeriez-vous à l'échelle pour traiter **1 million de tweets par seconde** ? Identifiez les goulots d'étranglement et proposez des solutions.

8 Livrables et rendu

8.1 Liste des livrables

#	Livrable	Description	Points
1	01_Streaming_Indicateurs	Notebook streaming complété	/5
2	02_ML_Sentiment_Model	Notebook ML complété	/5
3	03_Production_Pipeline	Notebook production complété	/4
4	dashboard	Notebook dashboard	/2
5	Modèle ML sur HDFS	<code>hdfs:///models/sentiment_model</code> (inclus)	
6	Questions ouvertes	Dans un notebook dédié ou dans les notebooks existants	/4
Total			/20

8.2 Modalités de rendu

- **Date de rendu** : *Dimanche 8 février 2026, 23h59* (heure de Paris).
- **Modalité** : Les notebooks doivent être laissés dans le répertoire `~/notebooks/` sur le cluster. Assurez-vous que :
 - Tous les notebooks sont **exécutables de bout en bout** (Kernel → Restart & Run All).
 - Le modèle est bien sauvegardé à l'emplacement indiqué sur HDFS.
 - Les réponses aux questions ouvertes sont rédigées clairement.

Rappel

Avant le rendu, vérifiez que vos notebooks s'exécutent correctement avec un **Restart & Run All**. Un notebook qui ne s'exécute pas entraîne une pénalité.

Annexes

A. Aide-mémoire Spark Structured Streaming

Opération	Syntaxe
Lecture socket	<code>spark.readStream.format("socket").option("host", ...).option("port", ...)</code>
Lecture Kafka	<code>spark.readStream.format("kafka").option("kafka.bootstrap.servers", ...)</code>
Fenêtre temporelle	<code>window(col, duration, slide)</code>
Watermark	<code>.withWatermark(col, delay)</code>
Modes de sortie	<code>append, complete, update</code>
Sinks	<code>console, memory, json, kafka</code>
Démarrage	<code>.writeStream.format(...).outputMode(...).start()</code>
Arrêt	<code>query.stop()</code>

```
df = spark.readStream \
    .format("socket") \
    .option("host", "master") \
    .option("port", 9999) \
    .load()

query = df.writeStream \
    .outputMode("append") \
    .format("console") \
    .start()
```

Exemple : lecture depuis un socket TCP

B. Aide-mémoire Apache Kafka

Concepts clés

Topic Canal de communication nommé dans lequel les messages sont publiés.

Partition Sous-division ordonnée d'un topic permettant le parallélisme.

Offset Position unique d'un message dans une partition.

Producer Application qui publie des messages dans un topic.

Consumer Application qui lit des messages depuis un topic.

Consumer Group Groupe de consumers qui se partagent la lecture des partitions.

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='kafka:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
producer.send("mon-topic", value={"message": "hello"})
producer.flush()
```

Producteur Kafka

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    "mon-topic",
    bootstrap_servers='kafka:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)
for msg in consumer:
    print(msg.value)
```

Consommateur Kafka

C. Commandes HDFS utiles

```
# Lister le contenu d'un repertoire
hdfs dfs -ls /path

# Afficher le contenu d'un fichier
hdfs dfs -cat /path/file | head

# Copier un fichier local vers HDFS
hdfs dfs -put local_file /hdfs/path

# Supprimer un fichier ou repertoire
hdfs dfs -rm -r /hdfs/path

# Afficher la taille d'un repertoire
hdfs dfs -du -h /path

# Creer un repertoire
hdfs dfs -mkdir -p /hdfs/path

# Telecharger un fichier depuis HDFS
hdfs dfs -get /hdfs/path/file local_file
```

Commandes courantes

Bon courage et bon projet !

Université de Poitiers — BUT 3 — 2025–2026